

PART 1: Core Java Foundations & OOP Mechanics

1.1 Strict Encapsulation

Definition & Internal Mechanics

Encapsulation is the bundling of data (fields) and the methods that operate on that data into a single unit (class), while restricting direct external access to the internal state. At the JVM level, when a field is declared `private`, the bytecode enforces that no external class's `getfield` or `putfield` instruction can reference it — the JVM's access control is enforced at class-loading and bytecode-verification time.

Reference Leak Problem: Even with `private` fields, if you return a reference to a mutable object, the caller can mutate the internal state. For example:

- Returning a raw `java.util.Date` exposes the internal reference
- Returning a raw `List` allows callers to `.add()` or `.remove()` without your class knowing

The fix: Return defensive copies.

Mutable vs. Immutable Objects:

- **Mutable:** State can change after construction (`StringBuilder`, `ArrayList`, `Date`)
- **Immutable:** State cannot change after construction (`String`, `Integer`, `LocalDate`)
- Immutable objects are inherently thread-safe since no synchronization is needed

Real-Life Analogy

Think of an ATM machine. You can check your balance and withdraw money (public methods), but you cannot directly reach into the machine's cash cassette or modify the internal ledger (private fields). The ATM's interface is the only way to interact with the underlying data — protecting the bank from unauthorized manipulation.

Interview Context & Why It Matters

Interviewers test this to verify you understand that `private` alone is insufficient for true encapsulation when mutable references are involved. Poor encapsulation leads to:

- **Data corruption** in multi-threaded environments
- **Hidden coupling** where external classes depend on internal representations
- **Violation of the Open/Closed principle**

Time complexity of defensive copies: $O(n)$ for collections, $O(1)$ for single objects.

Production-Ready Code Snippet

```
java
```

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.Date;
import java.util.List;
import java.util.Objects;

/**
 * Demonstrates strict encapsulation with defensive copying
 * to prevent reference leaks from mutable fields.
 */
public final class SecureBankAccount {

    private final String accountNumber;
    private double balance;
    private final List<String> transactionHistory;
    private Date lastModified;

    public SecureBankAccount(String accountNumber, double initialBalance) {
        if (accountNumber == null || accountNumber.isBlank()) {
            throw new IllegalArgumentException("Account number cannot be null or blank");
        }
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Initial balance cannot be negative.");
        }
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
        this.transactionHistory = new ArrayList<>();
        this.lastModified = new Date();
    }

    public String getAccountNumber() {
        return accountNumber; // String is immutable – safe to return directly
    }

    public double getBalance() {
        return balance; // Primitive – copied by value, no leak
    }

    /**
     * Defensive copy: returning a new Date prevents callers from
     * mutating our internal lastModified field via the returned reference.
     */
    public Date getLastModified() {

```

```

        return new Date(lastModified.getTime()); // DEFENSIVE COPY
    }

    /**
     * Returns an unmodifiable view of the transaction history.
     * Prevents external code from adding/removing entries directly.
     */
    public List<String> getTransactionHistory() {
        return Collections.unmodifiableList(transactionHistory); // DEFENSIVE VIEW
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive.");
        }
        this.balance += amount;
        this.lastModified = new Date();
        this.transactionHistory.add("DEPOSIT: +" + amount + " | Balance: " + balance);
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive.");
        }
        if (amount > balance) {
            throw new IllegalStateException("Insufficient funds. Balance: " + balance);
        }
        this.balance -= amount;
        this.lastModified = new Date();
        this.transactionHistory.add("WITHDRAWAL: -" + amount + " | Balance: " + balance);
    }

    // DEMONSTRATION OF THE LEAK vs. SAFE approach
    public static void main(String[] args) {
        SecureBankAccount account = new SecureBankAccount("ACC-001", 1000.0);
        account.deposit(500.0);

        // Attempt to tamper: get the Date reference
        Date modDate = account.getLastModified();
        modDate.setTime(0L); // This modifies only the COPY, not the internal state

        System.out.println("Internal date intact: " + account.getLastModified());

        // Attempt to tamper: get the List reference
    }

```

```

    List<String> history = account.getTransactionHistory();
    try {
        history.add("FRAUDULENT ENTRY"); // Throws UnsupportedOperationException
    } catch (UnsupportedOperationException e) {
        System.out.println("Transaction tampering blocked: " + e.getClass().getS
    }

    System.out.println("History size: " + account.getTransactionHistory().size()
}
}

```

Common Interview Question & Expert Answer

Q: "You have a class with a private `List<Transaction>` field. A colleague says it's fully encapsulated because the field is `private`. What would you say?"

A: "The field being `private` prevents direct field access, but the encapsulation is broken the moment the getter returns the raw reference. Anyone holding that `List` reference can call `.add()`, `.remove()`, or `.clear()` on it, mutating the class's internal state without its knowledge. True encapsulation requires one of three strategies: (1) return `Collections.unmodifiableList()` to prevent structural modification, (2) return a deep copy via `new ArrayList<>(this.transactions)`, or (3) avoid exposing the collection at all and instead expose computed results. Option (3) is the cleanest — it separates the representation from the interface. Additionally, if the `Transaction` objects themselves are mutable, a shallow copy is still insufficient — you'd need a deep copy where each `Transaction` is also copied."

1.2 Polymorphism — Compile-Time (Overloading)

Definition & Internal Mechanics

Method Overloading (compile-time or static polymorphism) allows multiple methods in the same class to share the same name but have different **method signatures** (different number, type, or order of parameters). The JVM resolves which method to call at **compile time** using static binding — the compiler writes the exact method reference into the bytecode's `invokestatic` or `invokevirtual` instructions based on the declared type.

Resolution Rules (in order):

1. Exact match
2. Widening primitive conversion (e.g., `int` → `long` → `float` → `double`)
3. Autoboxing (`int` → `Integer`)
4. Varargs (`int...`)

Important: The return type alone does NOT distinguish overloaded methods. The compiler would reject two methods with identical parameters but different return types.

Type Promotion Rules: When no exact match exists, Java applies widening — `byte` → `short` → `int` → `long` → `float` → `double`. Autoboxing has lower priority than widening, which can produce counterintuitive behavior.

Real-Life Analogy

Think of a customer service hotline with one phone number (method name). Depending on what you press (the "parameters" — "1 for billing", "2 for technical support"), the system routes you to a completely different department. The routing decision is made at the moment you dial (compile time), based on the inputs you provide.

Interview Context & Why It Matters

Overloading traps are common interview questions because developers often misunderstand the priority of widening over autoboxing. Getting this wrong in production leads to calling the wrong method silently with no runtime error — a particularly insidious class of bugs.

Production-Ready Code Snippet

```
java
```

```
/**
 * Demonstrates method overloading mechanics, type promotion,
 * and the widening-before-autoboxing resolution rule.
 */
public class OverloadingDemo {

    // Overload 1: exact int
    public static String describe(int value) {
        return "describe(int): " + value;
    }

    // Overload 2: long (widened from int if no exact match)
    public static String describe(long value) {
        return "describe(long): " + value;
    }

    // Overload 3: Integer (autoboxed from int – LOWER priority than widening)
    public static String describe(Integer value) {
        return "describe(Integer): " + value;
    }

    // Overload 4: Object (catch-all)
    public static String describe(Object value) {
        return "describe(Object): " + value;
    }

    // Overload 5: different parameter count
    public static String describe(int value, String label) {
        return "describe(int, String): " + label + "=" + value;
    }

    // Ambiguous overload – demonstrates the compile-time ambiguity problem
    // If you uncomment both of these, the call describe('A', 'B') becomes ambiguous
    // public static void ambiguous(int a, char b) {}
    // public static void ambiguous(char a, int b) {}

    public static void main(String[] args) {
        int primitiveInt = 42;
        Integer boxedInt = 42;

        // Exact match – calls describe(int)
        System.out.println(describe(primitiveInt));           // describe(int): 42
    }
}
```

```

// Widening beats autoboxing: int widens to long before boxing to Integer
// So if describe(int) didn't exist, this calls describe(long), NOT describe(int)
// Demonstrated by calling with a byte:
byte b = 10;
System.out.println(describe(b)); // describe(int) via widening

// Already boxed Integer: matches describe(Integer) exactly
System.out.println(describe(boxedInt)); // describe(Integer): 42

// Passing null - matches the MOST SPECIFIC applicable type
// Integer is more specific than Object, so describe(Integer) is chosen
System.out.println(describe((Integer) null)); // describe(Integer): null

// Varargs - lowest priority
System.out.println(describe(42, "value")); // describe(int, String): 42
}
}

```

Common Interview Question & Expert Answer

Q: "If you call `describe(42)` and there's both a `describe(long)` and a `describe(Integer)` overload but NO `describe(int)`, which one gets called and why?"

A: "Java calls `describe(long)` because widening primitive conversion has higher priority than autoboxing in the method resolution order. The compiler first tries to find an exact match, then applies widening conversions, and only then considers autoboxing. So `42` (an `int` literal) is widened to `long` rather than boxed to `Integer`. This is a classic interview trap — the priority order is: (1) exact type match, (2) widening, (3) autoboxing/unboxing, (4) varargs. This means `describe(Long)` would NOT be chosen for `describe(42L)` if `describe(long)` exists, because boxing a `long` to `Long` is lower priority than the exact primitive match."

1.3 Polymorphism — Runtime (Overriding)

Definition & Internal Mechanics

Method Overriding (runtime polymorphism) occurs when a subclass provides a specific implementation of a method already defined in a parent class. The JVM resolves which version to call at **runtime** using **dynamic dispatch** via the **Method Table (vtable)**.

How the vtable works:

- When a class is loaded, the JVM creates a **Method Table** for it — an array of method pointers

- Each entry points to the most-derived (most-specific) implementation of that method
- When `invokevirtual` executes, the JVM looks up the actual runtime type's vtable entry
- This happens in $O(1)$ time — a single pointer dereference

Rules for valid overriding:

- Same method name, same parameter list, same or covariant return type
- Access modifier must be same or more permissive (cannot reduce visibility)
- Cannot throw new or broader checked exceptions
- `static` methods cannot be overridden (they are shadowed/hidden instead)
- `private` and `final` methods cannot be overridden

`@Override` annotation: Instructs the compiler to verify that the method actually overrides a parent method. Without it, a typo in the method signature silently creates a new overloaded method instead of overriding — a dangerous and common bug.

Real-Life Analogy

Think of an employee handbook (the parent class) with a general policy for "expense approval". When a department head (subclass) joins the company, they override that policy with their department-specific procedure. When HR asks "how do you handle expense approval?" at runtime, they get the answer specific to that department head's actual type — not the generic policy.

Interview Context & Why It Matters

This is fundamental to understanding the **Liskov Substitution Principle (LSP)** — the 'L' in SOLID. Interviewers want to confirm you understand that the vtable exists, that `static` methods are NOT polymorphic, and that covariant return types are valid overrides.

Production-Ready Code Snippet

```
java
```

```

/**
 * Demonstrates runtime polymorphism via method overriding,
 * vtable dispatch, the @Override constraint, and
 * the difference between instance method overriding vs static method hiding.
 */
public class PolymorphismDemo {

    static abstract class Shape {
        // This will be in the vtable – dynamically dispatched
        public abstract double calculateArea();

        // This can be overridden – covariant return type allowed in subclass
        public Shape copy() {
            throw new UnsupportedOperationException("copy() not implemented");
        }

        // static methods are NOT in the vtable – they are hidden, not overridden
        public static String getCategory() {
            return "Shape";
        }

        @Override
        public String toString() {
            return getClass().getSimpleName() + " with area=" + String.format("%.2f",
        }
    }

    static class Circle extends Shape {

        private final double radius;

        public Circle(double radius) {
            if (radius <= 0) throw new IllegalArgumentException("Radius must be positive");
            this.radius = radius;
        }

        @Override // Compiler verifies this truly overrides – protects against typos
        public double calculateArea() {
            return Math.PI * radius * radius;
        }

        @Override
        public Circle copy() { // Covariant return type: Circle is a subtype of Shape
    }
}

```

```

        return new Circle(this.radius);
    }

    // This HIDES (not overrides) Shape.getCategory() – static binding
    public static String getCategory() {
        return "Circle";
    }
}

static class Rectangle extends Shape {

    private final double width, height;

    public Rectangle(double width, double height) {
        if (width <= 0 || height <= 0) throw new IllegalArgumentException("Dimension must be positive");
        this.width = width;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return width * height;
    }

    @Override
    public Rectangle copy() {
        return new Rectangle(this.width, this.height);
    }
}

public static void main(String[] args) {
    // Declared type is Shape – runtime type determines which calculateArea() runs
    Shape[] shapes = { new Circle(5.0), new Rectangle(4.0, 6.0), new Circle(3.0) };

    for (Shape shape : shapes) {
        // vtable dispatch: JVM looks up shape's actual runtime type
        System.out.println(shape); // calls overridden toString() → overridden code
    }

    // Demonstrating static method hiding vs. instance method overriding
    Shape shapeRef = new Circle(5.0);
    System.out.println("Instance method (dynamic): " + shapeRef.calculateArea());
    System.out.println("Static via Shape ref: " + Shape.getCategory()); // "Circle"
    System.out.println("Static via Circle ref: " + Circle.getCategory()); // "Circle"
}

```

```
// Covariant return type
Circle original = new Circle(3.0);
Circle copied = original.copy(); // No cast needed due to covariant return
System.out.println("Original == Copy? " + (original == copied)); // false -
}
}
```

Common Interview Question & Expert Answer

Q: "What happens at the JVM level when you call a method on a superclass reference pointing to a subclass object? And why can't `static` methods be overridden?"

A: "At the JVM level, `invokevirtual` is used for instance method calls. The JVM uses the actual runtime type of the object (found in the object header) to index into that type's vtable — the Method Table. It finds the pointer to the most-derived implementation and jumps to it. This is $O(1)$ since it's a direct array index lookup. The reference type (declared type) is irrelevant at this point — it only constrains which methods are visible at compile time. As for `static` methods: they belong to the class, not the object instance. There's no object to inspect for runtime type, so the compiler resolves `static` calls at compile time using `invokestatic` based on the declared class name. This is called method hiding — the subclass can declare its own `static` method with the same name, but they're entirely independent. Calling via a superclass reference always invokes the superclass's static method, regardless of what object (if any) the reference points to."

1.4 Abstraction — Interfaces vs. Abstract Classes

Definition & Internal Mechanics

Abstract Class: A class that cannot be instantiated, may contain a mix of abstract (unimplemented) and concrete (implemented) methods, can have instance fields, constructors, and any access modifier on members.

Interface: A contract specifying behavior. Historically only abstract methods, but since Java 8+:

- **default methods:** Concrete methods in interfaces with a body (for backward compatibility in evolving APIs)
- **static methods:** Utility methods tied to the interface type, not inherited by implementors
- **private methods (Java 9+):** Helper methods usable only within the interface's `default`/`static` methods
- **private static methods (Java 9+):** Same, but static

Functional Interface: An interface with exactly one abstract method (SAM — Single Abstract Method). These are the foundation of lambda expressions in Java. The `@FunctionalInterface` annotation enforces this constraint.

Key Differences Table:

Feature	Abstract Class	Interface
Instantiation	No	No
Multiple inheritance	Single	Multiple
Instance fields	Yes	No (only <code>public static final</code>)
Constructors	Yes	No
Default access modifier	<code>protected</code> / <code>public</code>	<code>public</code>
State	Yes	No (stateless by nature)
	<code>extends</code> / <code>implements</code>	<code>implements</code>

When to choose:

- **Abstract class:** When you have shared state, a partial implementation, and an "is-a" relationship (e.g., `Vehicle` as abstract with `fuelLevel` field)
- **Interface:** When you're defining a capability/contract that multiple unrelated classes can fulfill (e.g., `Serializable`, `Comparable`, `Runnable`)

Real-Life Analogy

An **interface** is like a job description — it specifies what skills (methods) you must have, but doesn't tell you how you got them. An **abstract class** is like an apprenticeship program — you get some training included (concrete methods), but you must complete certain modules yourself (abstract methods), and you can only be enrolled in one program at a time.

Interview Context & Why It Matters

The classic "interface vs abstract class" question is a filter for candidates who understand design principles. Interviewers want to hear about **multiple inheritance of type** (interfaces), **diamond problem avoidance**, **default method conflict resolution**, and **API evolution strategies**.

Production-Ready Code Snippet

```
java
```

```

import java.util.function.Predicate;

/**
 * Comprehensive demonstration of interfaces (with default, static, private methods)
 * vs abstract classes, functional interfaces, and conflict resolution.
 */
public class AbstractionDemo {

    // INTERFACE: defines a capability contract
    interface Auditable {
        String getAuditLog(); // abstract method – must be implemented

        default String getFormattedAuditLog() { // default method – optional override
            return "[AUDIT] " + getAuditLog(); // calls the abstract method
        }

        static Auditable noOp() { // static factory – not inherited
            return () -> "No audit trail available";
        }

        private String prefix() { // private helper – Java 9+
            return "AUDIT_V2: ";
        }
    }

    // INTERFACE: another capability
    interface Printable {
        void print();

        default String getFormattedAuditLog() { // SAME default method name – create conflict
            return "[PRINT_AUDIT] log";
        }
    }

    // ABSTRACT CLASS: provides shared state and partial implementation
    static abstract class Document {
        protected final String id;
        protected final String content;

        protected Document(String id, String content) {
            if (id == null || id.isBlank()) throw new IllegalArgumentException("Document ID cannot be null or blank");
            this.id = id;
            this.content = content != null ? content : "";
        }
    }

```

```

    }

    // Abstract – subclass must define how to render
    public abstract String render();

    // Concrete – shared implementation
    public String getSummary() {
        int preview = Math.min(content.length(), 50);
        return "Document[" + id + "]: " + content.substring(0, preview) + (content.length() > 50 ? "..." : "");
    }
}

// Concrete class: extends abstract class AND implements both interfaces
// Must resolve the default method conflict by overriding getFormattedAuditLog()
static class InvoiceDocument extends Document implements Auditable, Printable {

    private final double amount;

    public InvoiceDocument(String id, String content, double amount) {
        super(id, content);
        if (amount < 0) throw new IllegalArgumentException("Amount cannot be negative");
        this.amount = amount;
    }

    @Override
    public String render() {
        return "<invoice id='" + id + "' amount='" + amount + "'>" + content + "\n";
    }

    @Override
    public String getAuditLog() {
        return "Invoice " + id + " for $" + amount + " rendered";
    }

    @Override
    public void print() {
        System.out.println("Printing: " + render());
    }

    // MANDATORY override: resolves the conflict between Auditable and Printable
    // both having a default getFormattedAuditLog()
    @Override
    public String getFormattedAuditLog() {
        return Auditable.super.getFormattedAuditLog(); // explicitly delegate to superclass
    }
}

```

```

    }
}

// FUNCTIONAL INTERFACE usage
@FunctionalInterface
interface DocumentValidator {
    boolean validate(Document doc); // SAM

    default DocumentValidator and(DocumentValidator other) {
        return doc -> this.validate(doc) && other.validate(doc);
    }
}

public static void main(String[] args) {
    InvoiceDocument invoice = new InvoiceDocument("INV-001", "Services rendered

    System.out.println(invoice.render());
    System.out.println(invoice.getFormattedAuditLog()); // Uses Auditable.super
    System.out.println(invoice.getSummary()); // From abstract Document

    // Functional interface with lambda – leverages @FunctionalInterface SAM
    DocumentValidator notEmpty = doc -> !doc.content.isEmpty();
    DocumentValidator longEnough = doc -> doc.content.length() >= 5;
    DocumentValidator combined = notEmpty.and(longEnough); // default method com

    System.out.println("Valid? " + combined.validate(invoice)); // true

    // Standard library Predicate is also a functional interface
    Predicate<String> isLongId = s -> s.length() > 5;
    System.out.println("Long ID? " + isLongId.test("INV-001")); // true

    // Static factory from interface
    Auditable noOp = Auditable.noOp();
    System.out.println(noOp.getAuditLog());
}
}

```

Common Interview Question & Expert Answer

Q: "You have an interface with a `default` method, and a class that implements two interfaces both defining the same `default` method with the same signature. What happens, and how do you fix it?"

A: "This is the **diamond problem** adapted for Java interfaces. The compiler will flag a compile-time error: 'class X inherits unrelated defaults from interfaces A and B.' The implementing class is **required** to override the conflicting `default` method to resolve the ambiguity. Inside that override, you can delegate to a specific interface's implementation using the syntax `InterfaceName.super.methodName()` — for example, `Auditable.super.getFormattedAuditLog()`. This syntax was introduced specifically to handle this situation. If you're asking about abstract classes vs interfaces: if a class inherits a concrete method from a class and a default method from an interface with the same signature, the class method always wins — class hierarchy takes priority over interface default methods."

1.5 Inheritance Boundaries & the `final` Keyword

Definition & Internal Mechanics

The `final` keyword has three distinct applications:

1. **`final` variable:** The variable reference cannot be reassigned after initialization (but for reference types, the object the reference points to can still be mutated). For primitives, the value is a true constant. Instance `final` fields must be initialized in the constructor.
2. **`final` method:** Cannot be overridden in any subclass. The JVM can apply an optimization called **inlining** — substituting the method body at the call site — because it's guaranteed the method body never changes.
3. **`final` class:** Cannot be subclassed. Examples: `String`, `Integer`, `Math`. This enables the JVM to make aggressive optimizations and is critical for ensuring immutability.

Composition over Inheritance: The fragile base-class problem occurs when changes to a base class break subclasses in unexpected ways because the subclass depends on implementation details of the parent. The solution is to prefer **composition** — holding a reference to an object and delegating to it — over extending it. This is the 'L' and 'D' in SOLID (Liskov Substitution and Dependency Inversion).

Real-Life Analogy

`final` on a class is like a "recipe marked as proprietary" — no one can modify or extend it, and you always know exactly what you're getting. Composition is like being a restaurant that sources ingredients from specialists (a butcher, a baker) rather than trying to *be* a butcher and baker yourself — you stay focused on your role and delegate to experts.

Interview Context & Why It Matters

This topic tests SOLID principles knowledge. The key tradeoff: inheritance gives code reuse cheaply but creates tight coupling and the fragile base class problem. Composition requires more

boilerplate but results in more maintainable, testable code.

Production-Ready Code Snippet

```
java
```

```

import java.util.Objects;

/**
 * Demonstrates the final keyword and composition-over-inheritance.
 * Shows how inheritance creates fragile coupling vs composition's flexibility.
 */
public class FinalAndCompositionDemo {

    // PROBLEM: fragile base class via inheritance
    static class InstrumentedArrayListBAD extends java.util.ArrayList<String> {
        private int addCount = 0;

        @Override
        public boolean add(String e) {
            addCount++;
            return super.add(e);
        }

        @Override
        public boolean addAll(java.util.Collection<? extends String> c) {
            addCount += c.size(); // BUG: ArrayList.addAll() internally calls add()
            return super.addAll(c); // This double-counts! addCount will be inflated
        }

        public int getAddCount() { return addCount; }
    }

    // SOLUTION: composition – delegate to the List, don't extend it
    static final class InstrumentedList { // final: no further subclassing
        private final java.util.List<String> delegate; // final: reference won't change
        private int addCount = 0;

        public InstrumentedList(java.util.List<String> delegate) {
            this.delegate = Objects.requireNonNull(delegate, "Delegate list cannot be null");
        }

        public boolean add(String element) {
            addCount++;
            return delegate.add(element);
        }

        public boolean addAll(java.util.Collection<? extends String> elements) {
            addCount += elements.size();

```

```

        return delegate.addAll(elements); // delegate's addAll does NOT call our
    }

    public int size() { return delegate.size(); }
    public int getAddCount() { return addCount; }
}

// final class: cannot be subclassed – guarantees immutability
static final class ImmutablePoint {
    private final int x; // final field: must be set in constructor
    private final int y;

    public ImmutablePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }

    // final method: JVM can inline this – no vtable lookup needed
    public final double distanceTo(ImmutablePoint other) {
        Objects.requireNonNull(other, "Target point cannot be null");
        int dx = this.x - other.x;
        int dy = this.y - other.y;
        return Math.sqrt(dx * dx + dy * dy);
    }

    @Override
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
}

public static void main(String[] args) {
    // Demonstrating the fragile base class bug
    InstrumentedArrayListBAD badList = new InstrumentedArrayListBAD();
    badList.addAll(java.util.List.of("a", "b", "c"));
    System.out.println("Bad addCount (expected 3): " + badList.getAddCount());

    // Composition fix – correct count
    InstrumentedList goodList = new InstrumentedList(new java.util.ArrayList<>());
    goodList.addAll(java.util.List.of("a", "b", "c"));
    System.out.println("Good addCount (expected 3): " + goodList.getAddCount());
}

```

```

// final class usage
ImmutablePoint p1 = new ImmutablePoint(0, 0);
ImmutablePoint p2 = new ImmutablePoint(3, 4);
System.out.println("Distance: " + p1.distanceTo(p2)); // 5.0
}
}

```

Common Interview Question & Expert Answer

Q: "Why does Joshua Bloch say to prefer composition over inheritance? Can you give a concrete example where inheritance breaks unexpectedly?"

A: "Bloch's point is that inheritance violates encapsulation — the subclass depends on the *implementation details* of the superclass, not just its public contract. A classic example is `InstrumentedArrayList` extends `ArrayList`. If you override `add()` to count insertions and `addAll()` to also count, you'll get double-counting because `ArrayList.addAll()` internally calls `this.add()` for each element — and because of dynamic dispatch, it calls the overridden `add()`. This is an implementation detail of `ArrayList` you can't rely on staying stable across JDK versions. The fix is composition — hold a private `List` reference and forward calls to it. Since you're not extending `ArrayList`, your `addAll()` calling the delegate's `addAll()` will NOT trigger your overridden `add()`. The coupling is clean, the behavior is predictable, and future JDK changes to `ArrayList`'s implementation won't break your class."

1.6 The Object Contract — `equals()` and `hashCode()`

Definition & Internal Mechanics

The Contract (from the Java specification):

1. **Reflexivity:** `x.equals(x)` must return `true`
2. **Symmetry:** `x.equals(y)` $\leftrightarrow$ `y.equals(x)`
3. **Transitivity:** if `x.equals(y)` and `y.equals(z)`, then `x.equals(z)`
4. **Consistency:** Multiple calls return the same result if nothing changed
5. **Null safety:** `x.equals(null)` must return `false`

The `hashCode` contract:

- If `x.equals(y)` is `true`, then `x.hashCode() == y.hashCode()` must be `true`
- The converse is NOT required (hash collisions are allowed)

What breaks if you override `equals()` without `hashCode()`:

- `HashMap` and `HashSet` use `hashCode()` to find the bucket, then `equals()` to find the exact key
- If two logically equal objects have different hash codes, they land in different buckets
- `HashMap.get(key)` will return `null` even though an equal key was put in — a silent, catastrophic bug

Real-Life Analogy

Imagine a library catalog system. `hashCode()` is the Dewey Decimal Classification — it tells you which *shelf* to look on. `equals()` is the full examination of the book to confirm it's the exact one you want. If two identical books are filed under different Dewey codes, you'll never find the second one because you'll look on the wrong shelf first.

Interview Context & Why It Matters

This is one of the most frequently asked "gotcha" questions in Java interviews because breaking this contract causes silent bugs in `HashMap`, `HashSet`, `Hashtable`, and any collection using hashing. The consequences are $O(n)$ lookup degradation and data loss.

Production-Ready Code Snippet

```
java
```

```

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Objects;
import java.util.Set;

/**
 * Demonstrates the equals/hashCode contract, consequences of breaking it,
 * and production-quality implementations using Objects.hash() and
 * the instanceof pattern matching (Java 16+).
 */
public class EqualsHashCodeDemo {

    // BAD EXAMPLE: equals() overridden, hashCode() not – contract broken
    static class BrokenEmployee {
        private final String id;
        private final String name;

        public BrokenEmployee(String id, String name) {
            this.id = id;
            this.name = name;
        }

        @Override
        public boolean equals(Object obj) {
            if (this == obj) return true;
            if (!(obj instanceof BrokenEmployee)) return false;
            BrokenEmployee other = (BrokenEmployee) obj;
            return Objects.equals(id, other.id);
        }

        // hashCode() NOT overridden – uses Object's default (based on memory address)
    }

    // GOOD EXAMPLE: consistent, correct equals() + hashCode()
    static final class Employee {
        private final String id;    // primary business key
        private final String name;
        private final String department;

        public Employee(String id, String name, String department) {
            this.id = Objects.requireNonNull(id, "Employee ID required");
            this.name = Objects.requireNonNull(name, "Employee name required");
            this.department = Objects.requireNonNull(department, "Department required");
        }
    }

```

```

    }

    /**
     * Equality is based solely on business key (id).
     * Using Java 16+ pattern matching instanceof for cleaner cast.
     */
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;    // identity check: fast path
        if (!(obj instanceof Employee other)) return false; // type check + pattern matching
        return Objects.equals(id, other.id); // business key comparison
    }

    /**
     * Must use the same fields as equals() – only 'id' here.
     * Objects.hash() handles null safety and combines fields.
     */
    @Override
    public int hashCode() {
        return Objects.hash(id);
    }

    public String getId() { return id; }
    public String getName() { return name; }

    @Override
    public String toString() {
        return "Employee{id='" + id + "', name='" + name + "', dept='" + department + "'}";
    }
}

public static void main(String[] args) {
    // ---- Demonstrating broken contract ----
    BrokenEmployee e1 = new BrokenEmployee("E001", "Alice");
    BrokenEmployee e2 = new BrokenEmployee("E001", "Alice"); // logically equal
    System.out.println("e1.equals(e2): " + e1.equals(e2));    // true
    System.out.println("Same hashCode: " + (e1.hashCode() == e2.hashCode())); // false

    Set<BrokenEmployee> brokenSet = new HashSet<>();
    brokenSet.add(e1);
    System.out.println("BrokenSet contains e2: " + brokenSet.contains(e2)); // false

    Map<BrokenEmployee, String> brokenMap = new HashMap<>();
    brokenMap.put(e1, "Engineer");
}

```

```

        System.out.println("BrokenMap get e2: " + brokenMap.get(e2)); // null - cata

        System.out.println();

        // ---- Demonstrating correct contract ----
        Employee emp1 = new Employee("E001", "Alice", "Engineering");
        Employee emp2 = new Employee("E001", "Alice Updated", "Product"); // same id
        System.out.println("emp1.equals(emp2): " + emp1.equals(emp2)); // true - s
        System.out.println("Same hashCode: " + (emp1.hashCode() == emp2.hashCode()))

        Set<Employee> correctSet = new HashSet<>();
        correctSet.add(emp1);
        System.out.println("CorrectSet contains emp2: " + correctSet.contains(emp2))

        Map<Employee, String> correctMap = new HashMap<>();
        correctMap.put(emp1, "Engineer");
        System.out.println("CorrectMap get emp2: " + correctMap.get(emp2)); // "Engi
    }
}

```

Common Interview Question & Expert Answer

Q: "I have a class where I've overridden `equals()` to compare by `id` field, but I'm getting `null` back from a `HashMap` when I try to `get()` with a logically equal key. What's wrong?"

A: "You've overridden `equals()` but not `hashCode()`. The `HashMap` works in two steps: first it computes `hashCode()` to find the bucket index, then it traverses that bucket using `equals()` to find the exact entry. When you `put(key, value)`, the key's current `hashCode()` (which defaults to a memory-address-based value from `Object.hashCode()`) determines the bucket. When you create a new object with the same logical value and call `get()`, the new object has a *different* memory address and thus a different default `hashCode()` — so `HashMap` looks in the *wrong* bucket. It never even gets to call `equals()`. The fix is to override `hashCode()` using the same fields as `equals()` — only the `id` field in this case. A safe pattern is `return Objects.hash(id)`. The golden rule: objects that are `equals` must have identical `hashCode` values."

1.7 Deep vs. Shallow Copy — `Cloneable` vs. Copy Constructors

Definition & Internal Mechanics

Shallow Copy: A new object is created, but the fields are copied as-is. For primitives, the value is

copied. For object references, only the *reference* (memory address) is copied — both the original and copy point to the **same** underlying objects. Mutation of a shared mutable object via either reference affects both.

Deep Copy: A new object is created, and every mutable referenced object is also recursively copied so the original and copy are completely independent.

`Object.clone()` and `Cloneable`:

- `Object.clone()` performs a shallow copy
- To use it, a class must implement the `Cloneable` marker interface (has no methods)
- If `Cloneable` is not implemented, `clone()` throws `CloneNotSupportedException`
- `clone()` is `protected` in `Object` — must be overridden to `public` to expose it
- **Problem:** The `Cloneable` API is deeply flawed (Bloch, Item 13) — extralinguistic, fragile with final fields, and provides no deep copy guarantee

Copy Constructor (preferred): A constructor that takes an instance of the same class and copies its state. This is explicit, readable, works with `final` fields correctly, and doesn't require implementing a marker interface.

Real-Life Analogy

Shallow copy is like photocopying a document that has sticky notes on it — you get a copy of the paper, but the sticky notes (external references) are the same physical notes, visible to anyone who looks at the original too. Deep copy is like retyping the entire document AND handwriting new copies of every sticky note — completely independent.

Interview Context & Why It Matters

This directly impacts correctness in multi-threaded systems and anywhere defensive copying is required. The choice between shallow and deep copy has $O(1)$ vs $O(n)$ performance implications depending on the object graph depth.

Production-Ready Code Snippet

```
java
```

```
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

/**
 * Demonstrates shallow copy problems, deep copy solutions,
 * Cloneable pitfalls, and the preferred Copy Constructor pattern.
 */
public class CopyDemo {

    static class Address {
        String city;
        String country;

        public Address(String city, String country) {
            this.city = city;
            this.country = country;
        }

        // Copy constructor for Address
        public Address(Address other) {
            this.city = other.city;
            this.country = other.country;
        }

        @Override
        public String toString() {
            return city + ", " + country;
        }
    }

    // APPROACH 1: Cloneable (fragile – shown for comparison)
    static class PersonWithClone implements Cloneable {
        private String name;
        private Address address; // mutable field!
        private List<String> hobbies; // mutable field!

        public PersonWithClone(String name, Address address, List<String> hobbies) {
            this.name = name;
            this.address = address;
            this.hobbies = hobbies;
        }
    }
}
```

```

@Override
public PersonWithClone clone() {
    try {
        PersonWithClone copy = (PersonWithClone) super.clone(); // shallow c
        // Must manually deep-copy mutable fields:
        copy.address = new Address(this.address); // deep copy Address
        copy.hobbies = new ArrayList<>(this.hobbies); // new list (shallow e
        return copy;
    } catch (CloneNotSupportedException e) {
        // Should never happen since we implement Cloneable
        throw new AssertionError("Clone failed unexpectedly", e);
    }
}

public Address getAddress() { return address; }
public List<String> getHobbies() { return hobbies; }
public String getName() { return name; }
}

// APPROACH 2: Copy Constructor (preferred)
static final class Person {
    private final String name;
    private final Address address; // mutable – needs deep copy
    private final List<String> hobbies; // mutable list of immutable Strings

    // Primary constructor
    public Person(String name, Address address, List<String> hobbies) {
        this.name = Objects.requireNonNull(name);
        this.address = new Address(Objects.requireNonNull(address)); // defensive
        this.hobbies = new ArrayList<>(Objects.requireNonNull(hobbies));
    }

    // Copy constructor – explicit, clean, works with final fields
    public Person(Person other) {
        Objects.requireNonNull(other, "Source person cannot be null");
        this.name = other.name; // String is immutable
        this.address = new Address(other.address); // deep copy
        this.hobbies = new ArrayList<>(other.hobbies); // deep copy (Strings are
    }

    public Address getAddress() { return new Address(address); } // defensive co
    public List<String> getHobbies() { return java.util.Collections.unmodifiable
    public String getName() { return name; }
}

```

```

@Override
public String toString() {
    return "Person{name='" + name + "', address='" + address + "', hobbies=" +
}

}

public static void main(String[] args) {
    Address addr = new Address("New York", "USA");
    List<String> hobbies = new ArrayList<>(List.of("Reading", "Hiking"));

    // Test clone approach
    PersonWithClone original = new PersonWithClone("Alice", addr, hobbies);
    PersonWithClone cloned = original.clone();

    cloned.getAddress().city = "London"; // Modify clone's address
    cloned.getHobbies().add("Cycling"); // Modify clone's hobbies

    System.out.println("Original addr (should be New York): " + original.getAddr
    System.out.println("Original hobbies size (should be 2): " + original.getHob
    // Clone's changes did NOT leak to original – deep copy works

    System.out.println();

    // Test copy constructor approach
    Person p1 = new Person("Bob", new Address("Paris", "France"), List.of("Cooki
    Person p2 = new Person(p1); // copy constructor

    // p2's address is independent – we can't modify it via public API anyway (d
    System.out.println("p1: " + p1);
    System.out.println("p2: " + p2);
    System.out.println("Same? " + (p1 == p2)); // false
}
}

```

Common Interview Question & Expert Answer

Q: "What are the problems with Java's `Cloneable` interface, and why do most modern Java developers prefer copy constructors?"

A: "Joshua Bloch in *Effective Java* describes `Cloneable` as a deeply flawed API. First, `Object.clone()` only performs a **shallow copy** — you must manually deep-copy each mutable field, and there's no reminder or compiler assistance to do so. Second, `Cloneable` is a **marker interface** with no methods, so the contract is invisible — the behavior is determined by an

override of `Object.clone()` which is `protected` and has a strange activation mechanism. Third, it breaks with **final** fields pointing to mutable objects, since `clone()` creates the object via `super.clone()` without calling any constructor, which means `final` fields can't be reassigned after the shallow copy. Fourth, if a class in your hierarchy doesn't properly override `clone()`, the entire cloning chain breaks. Copy constructors solve all of these: they're **explicit** in their parameters, they work correctly with `final` fields (set them in the constructor), they're **readable** (you can see exactly what's being copied), they can take an **interface type** as the parameter (e.g., `new ArrayList<>(Collection c)` — works with any `Collection`), and they don't require implementing any interface."